Universidad de Las Palmas de Gran Canaria
Degree in Data Science and Engineering

Big Data. Academic year 2026/27

Search Engine Project

Stage 1: Building the Data Layer

Author	Isabel Hernández
Group	isahernande239h-prog (individual work)
Repository	https://github.com/isahernande239h-prog/stage_1
Date	November 2026

Contents

1. Introduction and goals 3
2. System architecture 3
3. Design decisions 4
    3.1. Downloading and splitting header and body 4
    3.2. Datalake organization 5
    3.3. Metadata 5
    3.4. Tokenizer and index model 6
    3.5. Inverted index structures 6
    3.6. Languages and equivalence 7
    3.7. How things were measured 7
4. Benchmarks and results 8
    4.1. Datalake 8
    4.2. Inverted index 11
    4.3. Comparing the languages 14
    4.4. Structure chosen for stage 2 14
    4.5. Limitations of the measurements 14
5. Conclusions and future work 15

1. Introduction and goals

In this project we have to build a search engine for Project Gutenberg books from scratch, based on an inverted index. This first stage is about the data layer, which is the base for everything that comes later: indexing, queries and services.

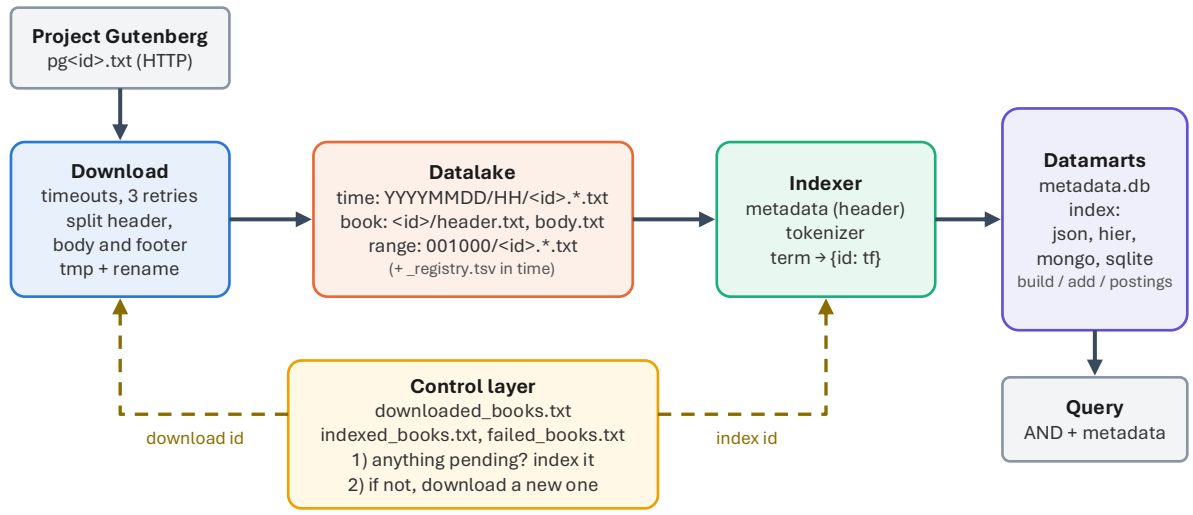
The data layer has two parts. The **datalake** keeps the text of each book as it arrives, but already split into header and body. The **datamarts** keep structured data that can be queried: the metadata of each book and the inverted index. On top of that there is a very simple **control layer** that keeps track of which books have been downloaded and which have been indexed.

These were the goals I set for this stage:

1. Download books by their ID, split the header, body and footer, and store them without ever accepting a download that was not written completely.
2. Compare three ways of organizing the datalake (by date and hour, by book and by ID ranges): write speed, cost of finding a book, detection of new books, recovery after a crash and disk space.
3. Get the title, author, language and year from the header and store them in SQLite.
4. Build the inverted index and compare four ways of storing it: a JSON file, a folder with one file per term, MongoDB and a SQLite table.
5. Write the control layer that coordinates downloading and indexing without repeating work.
6. Program everything in three languages (Python, Java and JavaScript) with the same rules, so that the comparison measures the language and not differences between the programs.

The REST API and the services are left for stage 2, although I already made some decisions with it in mind.

2. System architecture



Implemented three times with the same rules: Python 3.12, Java 17 and JavaScript (Node.js 24)

Figure 1. Architecture of the data layer. Solid arrows show the data flow and dashed arrows show the decisions of the control layer.

A book goes through these steps. First the file `pg<id>.txt` is downloaded, the line endings are normalized and the header (metadata and license), the body and the footer are split using the start and end lines (`*** START OF THE PROJECT GUTENBERG EBOOK` and the matching `END` line). The header and the body are written to the datalake as temporary files and then renamed. Only when both have been renamed is the ID added to `control/downloaded_books.txt`.

After that, the indexer takes the downloaded books that are not indexed yet. For each one it reads the metadata from the header and saves it in `datamarts/metadata.db`, tokenizes the body and adds its terms to the inverted index. When it finishes, it adds the book to `control/indexed_books.txt`. To answer a query, the book lists of each term are intersected and the metadata of each result is added.

The control layer follows the logic in the assignment. If there are downloaded books that are not indexed, it indexes the one with the lowest ID. If not, it downloads the next candidate that has not been downloaded and has not failed before. IDs that fail (because they don't exist or because they don't have the start and end lines) are written to `control/failed_books.txt` with the reason, so they are not tried again. When it starts, the program also runs a reconciliation: it deletes the temporary files and loose halves left by an interrupted write, adds to the control files any complete books that were missing, and removes the ones that are listed but no longer have files.

With the *range* layout and all the index structures, the data folder looks like this:

```
data/
  datalake/    001000/1342.header.txt, 1342.body.txt
  datamarts/   metadata.db
               inverted_index.json            (json)
               inverted_index/<A-Z|_>/*.txt   (hier)
               inverted_index.db              (sqlite; mongo uses the searchengine database)
  control/     downloaded_books.txt, indexed_books.txt, failed_books.txt
```

The three versions follow the same rules document (`spec/pipeline_spec.md`) and read the same files: the stop word list, the table of language codes, the test queries and the list of books in the corpus. They also have the same commands (`pipeline`, `search`, `ingest`, `bench-datalake`, `bench-index` and `canonical`) with the same options, so the benchmark script can run any of the three in the same way.

3. Design decisions

3.1. Downloading and splitting header and body

To download I use the URL given in the assignment, with my own User-Agent:

```
https://www.gutenberg.org/cache/epub/<id>/pg<id>.txt
```

There is a 10 s timeout to connect and 30 s to read. If there is a network error, an HTTP 5xx or a 429, the request is retried up to 3 times, waiting 1 s and then 2 s. A 404 means the book doesn't exist, so it is not retried. If the response has a `Content-Length` that doesn't match what arrived, the download is treated as incomplete and repeated. The text is decoded as UTF-8 and the BOM is removed.

The start and end lines are not the same in every book (THE or THIS, EBOOK or E-BOOK, with or without a space after the asterisks), so I look for them with a regular expression that accepts

those variants, and only at the start of a line. That way a row of asterisks inside the text is not mistaken for the end. The end line is only searched for after the start line, and if either of them is missing the book is rejected (`no_markers`) and doesn't go into the datalake. One detail that gave me trouble was trimming spaces at the start and end: JavaScript's `trim()` and Python's `strip()` also remove some Unicode spaces, and each language gave a different result. In the end only ASCII spaces are removed.

To avoid putting load on the Gutenberg server, I downloaded the benchmark corpus only once into a local cache, with a 1 s pause between requests (`scripts/download_corpus.py`). The candidates are a sample of the IDs from 1 to 75,000 taken with a fixed seed. 7 were discarded (4 didn't exist and 3 didn't have the start and end lines) and I kept the first 400 valid ones: 143 MB in 14 languages, 81% of them in English. The benchmarks read from that cache or from a small local HTTP server, `benchmarks/mirror_server.py`, which serves the same files with the same paths. The server is there to test the HTTP client of each language without depending on the Internet.

3.2. Datalake organization

I implemented the three organizations proposed in the assignment. All three write in the same way and only the path of each book changes:

Table 1. The three datalake organizations.

layout	path of a book	how a book is found	extra structure
time	`YYYYMMDD/HH/<id>.header.txt` (UTC hour of the write)	you need to know when it was downloaded	`_registry.tsv` (id and folder)
book	`<id>/header.txt` and `<id>/body.txt`	path computed from the id	none
range	`<bucket>/<id>.header.txt`, with `bucket = floor(id/1000)*1000` (`001000`)	path computed from the id	none

In *time* the path depends on when the book was downloaded and not on its ID, so a registry is needed to find a book without walking the whole tree. This layout also has a problem when it comes to measuring it: the benchmark downloads in a few seconds what a real crawler would take days to download, so with the real clock all the books would end up in the same folder and there would be no difference to see. That's why the benchmark uses a simulated clock that moves forward one hour every 10 books (40 folders for 400 books). The normal pipeline uses the system time.

Each book is always written in the same order: the body to a `.tmp` file, the header to another `.tmp`, the body is renamed, the header is renamed and, last of all, the book is added to the control file. If the process crashes, at most one book is left half written, and the reconciliation finds it and cleans it up. To check this I cause two kinds of crash (between the two renames, and after renaming but before writing to the control file) and resume with the clock 24 hours later, as if the program were started again the next day.

3.3. Metadata

The header is read line by line. A line like `Name: value` starts a field, and lines that start with a space continue it (for example `Most recently updated` under `Release date`). From there I get the title, all the authors (separated by semicolons), the language (the first one if there are several, turned into an ISO 639 code with a table), the original publication year (the last year in the

`Original publication` field) and the year the book was published on Gutenberg (the first year in `Release date`).

I store the two years separately because they don't mean the same thing, and also because many more headers have one than the other: only 59 of the 400 have the original year, while all of them have the Gutenberg date (and 377 have an author). If a value is missing it is stored as `NULL`, and I don't fill it in with another field. The SQLite `books` table has `book_id` as its primary key and I use `INSERT OR REPLACE`, so if a book is indexed twice it is not duplicated. The table also stores the paths of the two datalake files, so the text can be reached from the metadata.

3.4. Tokenizer and index model

The tokenizer lowercases the text, keeps the sequences of Unicode letters (`\p{L}`), drops the ones that are only one character long and removes 156 English stop words. Each language does this its own way: Python's `re` module has no `\p{L}`, so the character class is built with `unicodedata`, JavaScript needs the `u` flag and in Java I use `Character.isLetter`. Length is counted in Unicode characters and not in UTF-16 units, which in Java and JavaScript is not the same thing for some rare letters.

The stop word list is English only. A list that mixed languages would remove words that do mean something in another language (*die*, *son*), and 81% of the corpus is in English. Having one list per language is left as an improvement.

For each term, the index stores which books it appears in and how many times: `{book_id: tf}`. The frequency will be used in stage 2 to rank results by relevance. I left out positions because they would multiply the size of the index by the average number of times each term appears in each book (about 5 in this corpus), and they are not needed in this stage.

3.5. Inverted index structures

The four structures have the same interface (build the index, add a book and get the postings of a term), so the benchmark runs exactly the same code with all of them:

Table 2. Inverted index structures.

structure	format	adding a book
JSON	`datamarts/inverted_index.json`: `{"term": {"id": tf}}`	load, merge and rewrite the whole file (temporary file and rename)
Folders (*hier*)	`inverted_index/<letter>/<term>.txt`, with lines `id<TAB>tf`	append a line to the file of each term in the book
MongoDB	one document per term: `{_id: term, postings: {"id": tf}}`	`updateOne` with `$set` and `upsert`, in batches
SQLite	table `postings(term, book_id, tf)` with key `(term, book_id)`	`INSERT OR REPLACE` inside a transaction

JSON, folders and MongoDB are the three proposed in the assignment. I added SQLite myself, because the metadata was already in SQLite and it seemed the most logical candidate for stage 2. All four can cope with a book being indexed twice. The folder structure only appends lines, so there it is solved when reading: if a book appears several times in the same file, the last line wins.

The folder structure gave me two problems I didn't expect. First, `con.txt`, `nul.txt` and `aux.txt` are reserved names on Windows (and *con* is a word that shows up in the books). Second, the

Windows and macOS file systems are case-insensitive by default, so two Greek terms like *οδος* and *οδοσ* could end up in the same file. I solved it by keeping only the letters a-z as they are in the file name and encoding everything else as `%XX` (UTF-8 bytes). Reserved names get a _ added at the end.

3.6. Languages and equivalence

I chose Python 3.12 because it's the language of the examples in the assignment, Java 17 because it's the one that will be used for the stage 2 services (so I can reuse part of the code), and JavaScript with Node.js 24, which the assignment mentions and which works in quite a different way (V8 engine with JIT compilation). I ruled out C# because it didn't add much compared to Java.

To check that the three versions really do the same thing, the `canonical` command builds the metadata and the index (55 books at once and 5 added one by one) and writes the result in a fixed format, and the `equivalence` campaign compares the output of the three. The 12 combinations of language and structure give exactly the same 60 metadata records and the same 165,479 term lists (the SHA-256 digest is in `benchmarks/results/equivalence.txt`). On top of that, every benchmark checks that the 20 test queries return the same books.

Getting there took quite a lot of work, because some differences didn't cause any error and only showed up when comparing the outputs. These were the main ones:

- The `m` flag of JavaScript regular expressions also breaks lines at the `U+2028` character, so the text is read line by line, splitting only on `\n`.
- `JSON.parse` changes the order of keys that look like numbers, and in a plain JavaScript object the word *constructor* clashes with the prototype. That's why the JSON index in JavaScript uses a `Map` and its own reader.
- When there are bytes that are not valid UTF-8, Java puts a single replacement character where Python puts three. In that case Java uses its own decoder.
- Java and JavaScript sort strings by UTF-16 units and Python by Unicode code point, so all the outputs are sorted by code point so that they match.

There is one difference that can't be avoided: each language comes with a different version of Unicode (13 in Java 17, 15 in Python 3.12 and 17 in Node 24), and a few letters added in recent years only count as letters in some of them. None of them appear in the corpus.

3.7. How things were measured

All the measurements were taken on my laptop. Table 3 lists the machine, the versions of each runtime and how each thing was measured.

There are two things to keep in mind when reading the results. The first is that the files are always in the operating system cache, because they have just been written or read, so everything is measured with a warm cache (which is the normal situation for a service that is already running). The second is that Java's memory peak depends mostly on how the JVM manages memory: with a 3 GB maximum it is slow to release it, so the peak is not the minimum the program needs. Since the laptop only has 8 GB, I adjusted the limits so that no run had to use swap.

Table 3. Environment and measurement method.

Machine	MacBook Air with Apple M3 (8 cores: 4 performance and 4 efficiency), 8 GB of RAM and internal SSD (APFS)
Operating system	macOS 26.3.1 (arm64)
Runtimes	Python 3.12.13 (SQLite 3.53); OpenJDK 17.0.21 (Temurin) with `-Xms256m -Xmx3g`; Node.js 24.15 with `--max-old-space-size=3072`; local MongoDB 7.0.14 with 1 GB of cache
Corpus	400 books (143 MB), always in the same order; subsets of 25 to 400 books
Repetitions	datalake 5, recovery 3, local HTTP server 3, index 3; the runs are interleaved
Isolation	each configuration in a new process and with an empty working folder
Times	monotonic clock inside each program (`perf_counter`, `nanoTime`, `hrtime`)
Memory	peak resident memory of the process, sampled every 20 ms from outside with psutil; with MongoDB, the `mongod` server too
Disk	file bytes and bytes used on disk; for MongoDB, `storageSize + totalIndexSize` after an `fsync`

4. Benchmarks and results

The numbers are the mean ± the standard deviation of the repetitions. The data of every run is in `benchmarks/results/raw/`, and the script `benchmarks/analyze.py` computes the full tables from it (`benchmarks/results/tables.md`). Here I only include the most important ones.

4.1. Datalake

Writing

Table 4. Writing the 400 books (139 MB) from the local cache, 5 repetitions.

language	time (books/s)	book (books/s)	range (books/s)	MB/s (range)	peak RAM (MB)
Python	378 ± 15	395 ± 10	399 ± 12	138 ± 4	103-104
Java	468 ± 13	488 ± 7	469 ± 14	163 ± 5	331-334
JavaScript	567 ± 5	576 ± 2	572 ± 8	198 ± 3	211-225

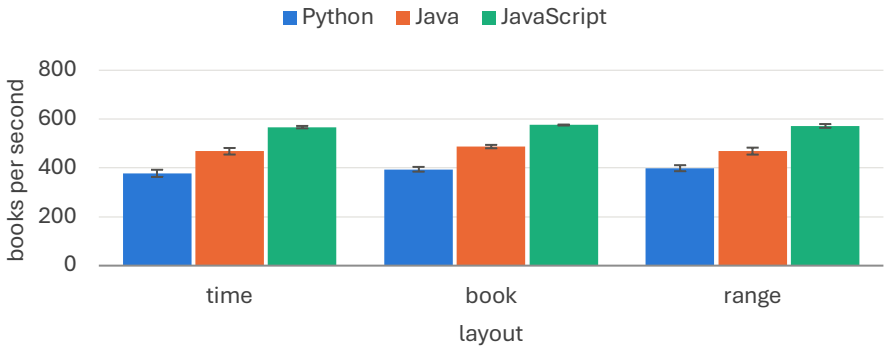


Figure 2. Books per second when writing the 400 books, by layout and language. The error bars are the standard deviation.

The layout hardly changes the write speed: between *time*, *book* and *range* there is less than a 5% difference, which is about the size of the standard deviation. This makes sense, because in all three cases each book means two temporary files and two renames, and only the target folder changes (plus one more line in the registry for *time*). What does change is the language: JavaScript writes about 570 books per second, Java about 470 and Python about 390. In all three the time goes into reading the file, finding the start and end lines and writing about 350 KB per book, so the difference has to come from how each language processes text and how much each call to the file system costs.

If the books arrive over HTTP from the local server, the speed drops by 23 to 34% (Python 292-300, Java 318-328 and JavaScript 395-413 books/s), which is the cost of each language's HTTP client. Downloading for real from gutenberg.org (only 10 books and one run, so it's just a rough figure) gives between 2.6 and 3.8 books/s, about a hundred times less. In other words, in a real crawler the network and the pause between requests are what matter, and the write speed is not a reason to choose one layout over another.

Finding a book



Figure 3. Time to find a book with 400 books in the datalake: with the computed path (or the registry in time) and walking the tree.

With the computed path (*book* and *range*) or with the registry already loaded (*time*), finding a book takes between 4 and 13 µs, which is basically checking that the two files exist. The difference is that *time* has to read the registry first, which takes between 0.1 and 0.5 ms with 400 books and grows with the number of books. And if the registry is lost or gets out of date, there is no direct way to find anything anymore. Without any help you have to walk the tree: 1.5-2.3 ms in *time*, 1.9-2.7 ms in *range* and 5.9-7.5 ms in *book*, which is the slowest because it has to open 400 folders, one per book.

Detecting new books

Table 5. Detection of the 10 new books with 400 books in the datalake (ms, average of the three languages). Every run found exactly the 10 books.

layout	control files	full scan	pruned scan (time only)
time	0.26 ± 0.11	1.71 ± 0.28	0.21 ± 0.05
book	0.44 ± 0.23	6.38 ± 0.64	-
range	0.35 ± 0.21	2.17 ± 0.28	-

Comparing the two control files takes less than half a millisecond and barely depends on the size. Walking the whole datalake grows linearly with the number of books, and once again it is more expensive in *book*. This is the only advantage of *time*: if you know which folder the last indexed book was in, you only need to look at the folders after it, and the cost stays at about 0.2 ms whatever the size. But that only works if new books always land in new folders (the clock never goes backwards and nothing is rewritten in an old folder), and the control layer already solves the same problem, faster and without those conditions.

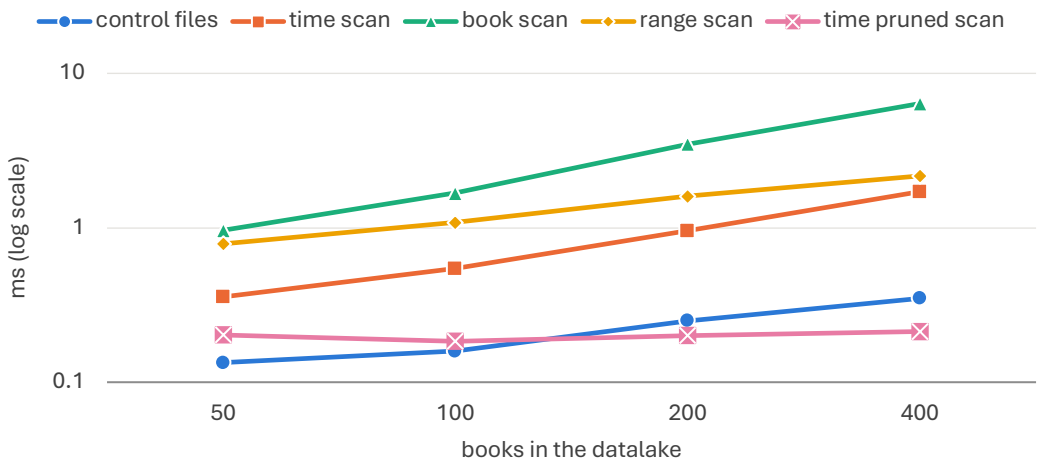


Figure 4. Time to find the 10 books that are downloaded but not indexed, depending on the size of the datalake (average of the three languages, log scale).

Recovery after a crash

Table 6. Runs that end with a consistent datalake (3 languages x 3 repetitions) when the process is killed at book 51 of 100 and resumed 24 hours later.

layout	crash	control only	with reconciliation	reconcile in ms (Py / Java / JS)
time	between the two renames	0/9 (1 orphan, 1 temporary)	9/9	0.7 / 8.9 / 2.4
time	before writing to the control file	0/9 (1 duplicate)	9/9	0.8 / 9.1 / 1.7
book	both	9/9	9/9	1.0-1.2 / 4.9-5.4 / 1.9-2.1
range	both	9/9	9/9	0.9-1.0 / 4.9-5.6 / 1.9-2.0

Of all the tests, this is the one where the layouts differ the most. In *book* and *range* the path of a book depends only on its ID, so when the program resumes, the interrupted book is written again in the same place and replaces whatever was left half done: the datalake ends up fine even without reconciling. In *time* the path depends on the hour, and when the program resumes the next day the book goes to a different folder. If the crash happened between the two renames, the old half is left orphaned; if it happened before writing to the control file, the book ends up stored twice. With the reconciliation at startup, the three layouts end up fine in every run, and since it takes less than 10 ms, it is always done. Java takes a bit longer to reconcile, probably because that code only runs once and the JIT doesn't have time to optimize it.

Disk space

With 400 books the three layouts store the same 800 files and 138.75 MB (140.81 MB used on disk, 1.5% more because of rounding up to whole blocks). What changes are the folders and the extra structures: *time* uses 42 folders (2 days and 40 hours) and a 7 KB registry, *book* uses 400 folders and *range* uses 75. The control files take 4.6 KB in all three cases, and the numbers are the same in the three languages.

Chosen layout

I'm going with *range*. It finds a book with a computed path, without any registry to maintain, it recovers by itself after a crash and it has few folders: 75 against the 400 of *book*, and with the full catalog of about 75,000 books it would be 75 folders of 1000 books against 75,000 folders.

Walking it is also almost as cheap as walking *time*. The *book* layout recovers just as well but creates too many folders, and *time* only wins at detecting new books, which the control layer already takes care of. It is also the layout I'll use in stage 2.

4.2. Inverted index

Size of the problem

Table 7. Size of the index depending on the number of books (after removing stop words).

books	tokens	distinct terms	postings	same answers in the 12 combinations
25	637,291	60,383	136,859	yes
50	1,586,937	124,779	319,297	yes
100	3,305,997	238,601	656,193	yes
200	6,484,063	332,364	1,272,929	yes
400	12,973,396	528,165	2,460,831	yes

The postings grow almost in proportion to the number of books, but the distinct terms don't: with 16 times more books there are 8.7 times more terms, because new books mostly repeat words that were already there. In every run, the 20 queries in `spec/queries.txt` return the same books with the three languages and the four structures.

Build, queries, disk and memory

Table 8. Full index of the 400 books, 3 repetitions. The queries are the 20 from the specification repeated 5 times. For hier, disk means bytes used on disk. The rest of the deviations are in tables.md.

structure	language	build (s)	tokenize (s)	save (s)	open (ms)	query p50 (ms)	query p95 (ms)	disk (MB)	peak RAM (MB)
json	Python	27.07 ± 0.05	25.69	1.38	387	0.039	0.059	32.3	501 ± 31
json	Java	3.05 ± 0.05	2.72	0.33	348	0.054	0.111	32.3	1091 ± 51
json	JavaScript	4.40 ± 0.20	4.12	0.28	484	0.039	0.058	32.3	841 ± 13
hier	Python	73.38 ± 0.75	25.66	47.72	3.6	0.133	0.188	2163	332 ± 6
hier	Java	43.17 ± 0.92	2.80	40.37	2.6	0.150	0.393	2163	941 ± 9
hier	JavaScript	45.01 ± 0.74	4.50	40.51	2.3	0.113	0.167	2163	657 ± 31
mongo	Python	27.95 ± 0.20	25.79	2.16	-	0.268	0.346	29.1	374 + 205 (mongod)
mongo	Java	5.12 ± 0.24	2.74	2.38	-	0.535	0.942	31.7	1090 + 218 (mongod)
mongo	JavaScript	6.06 ± 0.25	4.14	1.92	-	0.423	0.549	29.1	812 + 244 (mongod)
sqlite	Python	27.65 ± 0.23	25.60	2.04	0.2	0.105	0.153	50.0	335 ± 10
sqlite	Java	5.20 ± 0.22	2.74	2.47	1.0	0.157	0.217	50.0	1021 ± 78
sqlite	JavaScript	6.32 ± 0.24	4.18	2.13	0.2	0.149	0.221	50.0	703 ± 42

Build. Except in *hier*, almost all the time spent building the index goes into tokenizing: in Python it's 25.7 s out of about 27-28 s in total (Figure 5). Python tokenizes roughly 0.5 million tokens per second, JavaScript about 3 million and Java up to 4.7 million (Figure 6). Java gets faster the bigger the corpus is, probably because the JIT needs a while to optimize the tokenizer loop, while JavaScript runs at the same speed from the start. Saving the index is cheap in JSON (0.3 s in Java and JavaScript and 1.4 s in Python, which serializes more slowly) and takes about 2 s in MongoDB and SQLite, almost the same in the three languages because the database does the work. The *hier* structure is a special case: it has to create 528,165 files, one per term, and takes 40-48 s whatever the language. That's about 80 µs per file, which is what creating a file costs on APFS.

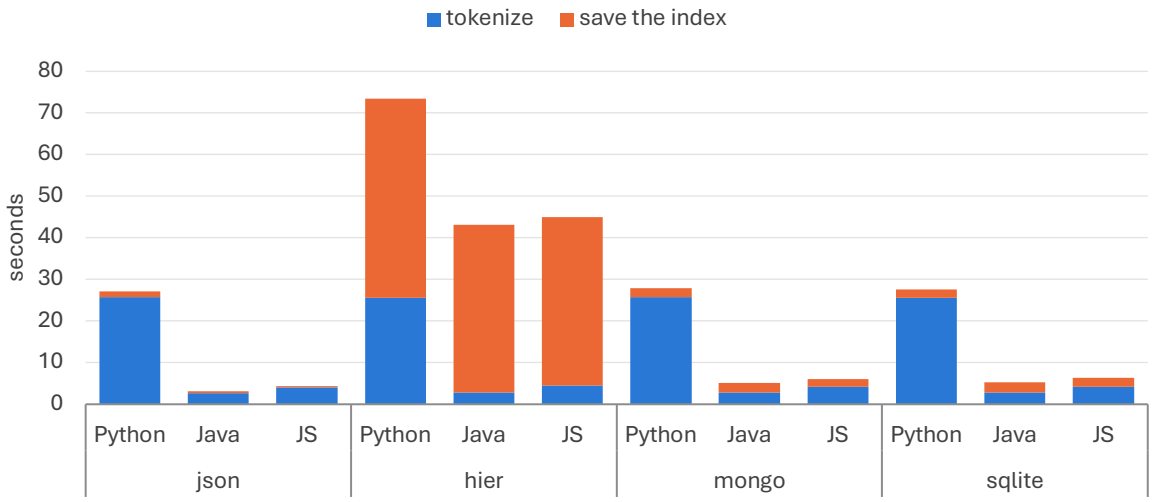


Figure 5. Index build time with 400 books, split into tokenizing and saving, by structure and language.

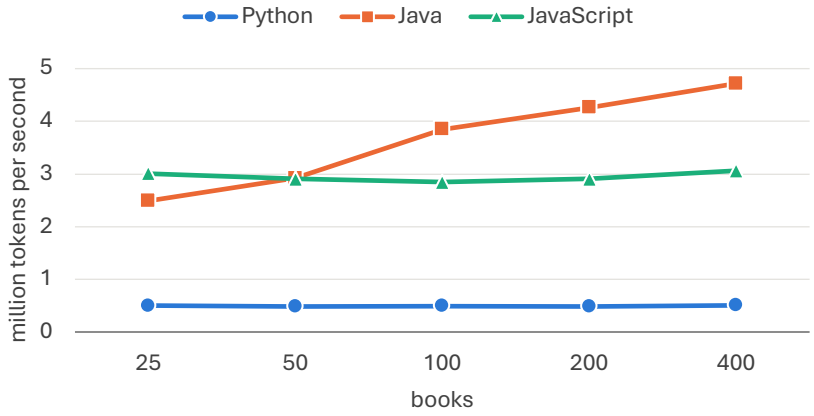


Figure 6. Tokenizer speed (millions of tokens per second) depending on the size of the corpus.

Queries. All the structures answer in less than a millisecond (Figure 7). JSON is the fastest (0.04-0.05 ms) because it keeps the whole index in memory, but it has to be loaded first, and opening the 32 MB file takes between 350 and 480 ms and uses quite a lot of memory. SQLite and *hier* take 0.1-0.16 ms (a lookup in the B-tree or reading a small file per term), and MongoDB 0.27-0.54 ms, because each term is a request to the server. In all of them the time goes up with the size, because the lists of the frequent terms get longer.

Disk. JSON and MongoDB take about the same space (29-32 MB, because MongoDB compresses the data), SQLite a bit more (50 MB, because it repeats the term in every row) and *hier* 2.16 GB (Figure 8). The *hier* files are a few dozen bytes on average, but each one takes at least a 4 KB block, so in the end the index takes up 67 times more space than the JSON file.

Memory. The peak depends mostly on the language: 330-500 MB in Python, 660-840 MB in JavaScript and 940-1090 MB in Java, which releases memory late with a heap of up to 3 GB. Within each language, JSON uses the most, because besides the index in memory it builds the whole text before writing it. With MongoDB you have to add another 200-250 MB for the `mongod` process.

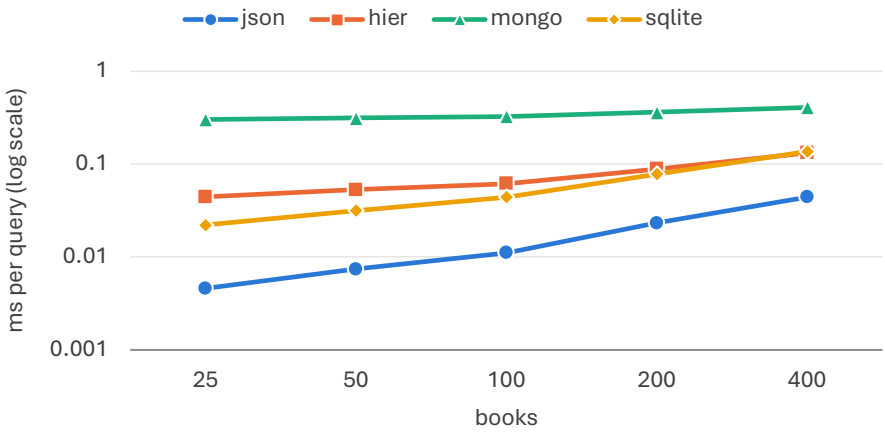


Figure 7. Median time per query depending on the number of books (average of the three languages, log scale).

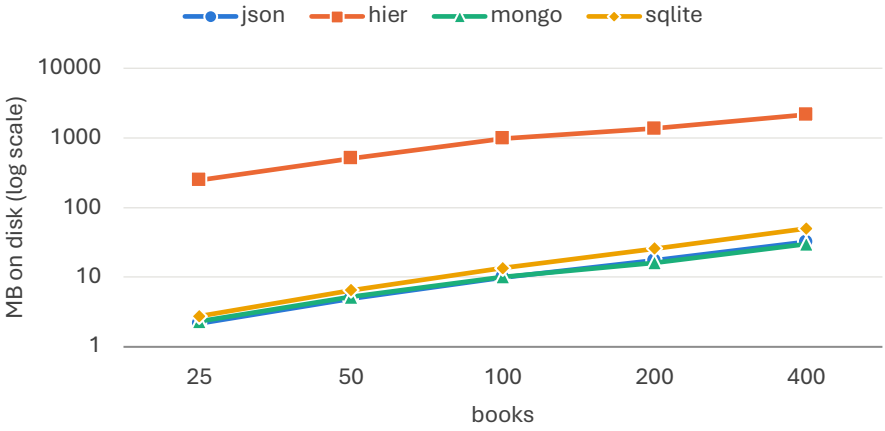


Figure 8. Disk space of the index depending on the number of books (log scale; it is almost the same in the three languages).

Incremental updates

Table 9. Cost of adding 10 books to an index that already has n books, in ms per 1000 distinct terms of the added books.

structure	language	n = 25	n = 50	n = 100	n = 200
json	Python / Java / JS	18.1 / 3.6 / 4.7	21.0 / 4.2 / 6.0	56.3 / 11.8 / 14.8	126.4 / 24.7 / 29.9
hier	Python / Java / JS	130 / 114 / 118	141 / 119 / 118	154 / 152 / 150	176 / 163 / 165
mongo	Python / Java / JS	15.6 / 16.2 / 15.3	15.6 / 15.9 / 15.5	16.6 / 17.5 / 16.5	17.5 / 19.2 / 17.7
sqlite	Python / Java / JS	2.6 / 2.9 / 2.8	3.5 / 3.9 / 3.6	5.8 / 6.5 / 5.7	8.2 / 9.0 / 8.3

For stage 2 this is the most important measurement, because books will arrive one at a time. In JSON the whole file has to be loaded, merged and rewritten, so the bigger the index, the more it costs: with 200 books Python already takes 126 ms per 1000 terms, and a book has about 7000 distinct terms on average. The *hier* structure has a cost that is more or less fixed but high, because it opens and closes one file per term. MongoDB stays at about 16 ms and SQLite is the cheapest (3-9 ms), although it goes up a little as the B-tree grows. In these three the language makes almost no difference.

4.3. Comparing the languages

Table 10. Summary of the comparison between languages with 400 books.

	Python	Java	JavaScript
Lines of code (without tests or blank lines)	1095	2580	1565
Writing to the datalake with range (books/s)	399	469	572
Tokenizing the 400 books (s)	25.7	2.7	4.1
Saving the index in SQLite / MongoDB (s)	2.0 / 2.2	2.5 / 2.4	2.1 / 1.9
Creating the hier files (s)	47.7	40.4	40.5
Peak memory building with SQLite (MB)	335	1021	703

The language shows the most in the work that depends on the CPU, above all tokenizing: Python is between 6 and 9 times slower than Java and JavaScript. In the work that depends on the disk or on another process (writing to the datalake, saving to MongoDB or SQLite, or creating thousands of files) the three perform about the same. Python was the most comfortable to program and the one that uses the least memory. Java needs the most code, but it's the fastest once the JIT gets going, and I'll be able to reuse it in stage 2. JavaScript sits in the middle in almost everything, although it was the one that gave me the most trouble to make its output match the other two (key order in `JSON.parse`, object prototypes and UTF-16 strings).

4.4. Structure chosen for stage 2

For stage 2 I'm choosing SQLite. It's the best at adding books one at a time, it answers in a tenth of a millisecond, it doesn't need a separate server and it can save the postings and the metadata of a book in the same transaction. That last point is exactly what will be needed so that the search service never sees a half-indexed book. Its weak point is that it only allows one writer at a time, which will have to be taken into account in stage 2. JSON only makes sense for a small index that doesn't change, *hier* is ruled out because of the disk space and the build time, and MongoDB would make sense if the index were spread over several machines, which is what stage 3 is about.

4.5. Limitations of the measurements

- Everything was measured on a single laptop and with a warm system cache. The M3 has two kinds of cores and the system decides where each process runs, which adds some noise. That's why I interleaved and repeated the runs, and most of the deviations are below 5%.
- Three repetitions for the index are enough for the big differences I discuss, but not for differences of a few percent like the ones between layouts when writing.
- Memory is sampled every 20 ms, so a shorter peak could be missed.
- The test against gutenberg.org is only 10 books and a single run, and it depends on the network.
- 400 books are a bit more than 0.5% of the catalog. The trends can be extrapolated, but JSON and *hier* would be even worse with the full catalog.

5. Conclusions and future work

The data layer works the same way in the three languages: with the same corpus, the 12 combinations of language and structure give the same metadata, the same index and the same answers. The main things I've learned in this stage are:

- The three layouts write at almost the same speed, so the choice depends on what happens when something fails. The *range* and *book* layouts recover by themselves after a crash because the path only depends on the ID, while *time* needs a registry and a reconciliation step so it doesn't leave duplicates or orphans. Between *range* and *book* I chose *range* because it creates far fewer folders.
- With the control layer, detecting pending books takes less than half a millisecond (it's just two text files) and doesn't depend on the layout. Since the control file is only written after the rename, an incomplete book is never marked as downloaded.
- For the index, the most important thing is being able to add books one at a time. JSON is the fastest for queries, but every update rewrites the whole file, and *hier* is very simple but takes up 67 times more disk space. SQLite works best overall and it's the one I'm going to use in stage 2.
- The language matters most in the CPU-heavy work: Python is between 6 and 9 times slower at tokenizing. When writing to the datalake, SQLite or MongoDB it keeps a similar pace to the other two.
- Getting the three programs to give exactly the same output took more effort than I expected. Trimming spaces, sorting strings or decoding UTF-8 changed the output without any error, and it only showed up when comparing the canonical output of the three. Something similar happened with the measurements: MongoDB's `collStats` returned almost 0 until I added an `fsync` before measuring.

Things to improve later on:

- Stop word lists per language and *stemming*, since 19% of the corpus is not in English.
- Storing positions (or at least blocks) to be able to search for exact phrases.
- Compressing the postings lists (gaps between IDs and variable-length integers).
- A version of *hier* that groups terms by prefix instead of using one file per term.
- Tokenizing in parallel, especially in Python, where one process per core would cut the build time quite a lot.
- Repeating the measurements with a cold cache and with the full catalog, and comparing other metadata stores (this is optional in the assignment and I haven't done it).